

The 56 Hooks That Keep Claude Code From Going Rogue

Free guardrail pack from @build.inquiet

Claude Code will confidently tell you it did something it didn't, edit a file it shouldn't, or burn tokens on work a cheaper tool should do. Hooks make that impossible. They run automatically on every tool call and the agent cannot bypass them. Here's the system I run. Steal it.

A hook is a small script that fires on a Claude Code event (before a tool runs, after it runs, when the turn ends, when a session starts). It can warn, block, or inject a reminder. You wire them in `settings.json`. That's the whole idea: the rules enforce themselves instead of living in a doc the agent forgets.

The 7 jobs these hooks do

1. *Protect files from the agent (write guards)*

- zone-guard** — block any write to a protected file before it lands. No prompt, no undo.
- bash-zone-guard** — same protection, but for writes attempted through shell commands.
- plan-mode-guard** — no file writes while you're still planning.
- plan-archiver** — snapshot a plan file before it gets overwritten.
- batch-counter** — hard cap on write-ops per batch so a runaway loop can't rewrite your repo.

2. *Stop it lying about being done (verification gates)*

This is the big one. The agent's worst habit is declaring success it never checked.

- stop-verification** — block the turn from ending on an unverified success claim with no evidence shown.
- claim-provenance** — flag state claims (counts, status) that have no source cited right next to them.
- closing-integrity** — re-derive any numeric/state claim in a summary from a live command, not memory.
- api-verify-reminder** — after any external API call, force a fresh check that it actually worked.
- capitulation-sentinel** — block reflexive agreement that has no evidence behind it. Truth over politeness.

3. *Keep your money and tokens safe (cost + routing)*

- cost-gate** — estimate the cost before a paid API call, surface it before it fires.
- routing-advisor** — push bulk/mechanical work to a cheaper model instead of the expensive one.

bulk-burn-tracker — log when the expensive model did work a cheaper tool should have.

batch-counter — also caps runaway batches (see #1).

context-pressure — watch the context window and auto-save before it compresses and loses state.

4. *Never leak a secret (safety scanners)*

secret-leak-detector — catch API keys/tokens before they get written into a file.

secret-output-guard — block any command that would dump your environment (and its secrets) to the screen or logs. Fails closed.

anonymity-scan — flag real names/identifying info before they end up in something shareable.

5. *Keep state honest (sync gates)*

memory-sync — keep the agent's memory and your repo's source-of-truth in step.

state-sync sentinels — block the turn from ending if you did the action but never updated the file that tracks it. No silent drift.

lint + save gates — validate your single-source-of-truth file's schema before any write lands.

6. *Run a clean session (lifecycle)*

session-start — print the date, open reminders, and anything carried over from last time, every session.

commit-nudge — suggest a commit at natural checkpoints so work doesn't pile up uncommitted.

transcript-enforcer — make sure the session actually gets logged.

context-pressure — trigger a save before the window fills (see #3).

7. *Enforce your own SOPs (procedural gates)*

pre-read sentinels — block a task until the agent has actually read the rules/spec it's supposed to follow this session.

deprecated-source-guard — refuse reads of files you've marked stale, and name the current one instead.

approval gates — block a publish/ship action until a fresh human-approval token exists.

output gates — refuse a publish whose payload is malformed (empty media, wrong format) before it hits the platform.

Start with these 5

Don't build all 56 at once. These five catch the most damage for the least effort.

1. **stop-verification** — kills the lying-about-done problem. Highest value, build it first.
2. **zone-guard** — protects the files you can't afford to lose.

3. **secret-output-guard** — one leaked key ruins your week.
4. **cost-gate** — stops surprise API bills.
5. **session-start** — gives every session its bearings.

The one that changes everything (copy-paste starter)

A minimal "block writes to protected paths" hook. Save as `.claude/hooks/zone-guard.py`:

```
#!/usr/bin/env python3
import json, sys, os

PROTECTED = ("CLAUDE.md", ".env", "secrets/", "settings.json")

data = json.load(sys.stdin)
path = data.get("tool_input", {}).get("file_path", "")
rel = os.path.relpath(path) if path else ""

if any(rel == p or rel.startswith(p) for p in PROTECTED):
    print(f"BLOCKED: {rel} is protected. Get explicit approval before editing.", file=sys.stderr)
    sys.exit(2)  # exit code 2 = block the tool call
sys.exit(0)
```

Wire it in `settings.json`:

```
{
  "hooks": {
    "PreToolUse": [
      { "matcher": "Edit|Write", "hooks": [
        { "type": "command", "command": "python3 .claude/hooks/zone-guard.py" }
      ]}
    ]
  }
}
```

That's it. Now Claude Code physically cannot write to those files without you clearing it first.

Want the full system? Follow [@build.inquiet](#). I post one Claude build a week, and the rest of these hooks (with the code) ship there.